EventForce

Event Management System

A Customer Relationship Management application on the Salesforce platform

Project Title	EventForce — Event Management System
Platform	Salesforce (Lightning Experience)
Submitted by	[Your Name]
Roll Number	[Roll Number]
Department	[Department]
Guided by	[Guide Name]
Institution	[College Name]
Academic Year	[2025 – 2026]

Replace the bracketed placeholders on this page before submission.

A Customer Relationship Management application on the Salesforce platform

Project Title	EventForce — Event Management System
Platform	Salesforce (Lightning Experience)
Environment	Salesforce Developer Edition Org
Language / Tooling	Apex, SOQL, Flow Builder, Process Builder-free declarative automation
Category	Cloud CRM / Academic mini-project

Abstract

Event management companies coordinate four separate parties — clients, vendors, venues and internal coordinators — usually across disconnected spreadsheets and email threads. Booking conflicts, missed client reminders and unapproved cancellations are the predictable consequences.

EventForce is a cloud CRM built on the Salesforce platform that brings all four parties into a single integrated system. It models the event lifecycle as six custom objects connected through lookup, master-detail and many-to-many relationships, and automates the repetitive work: a venue is reserved and released automatically as events are booked and cancelled, clients receive a reminder three days before their event, cancellation requests are routed through a formal approval process with email notification to both approver and client, and past events are marked complete by a nightly batch job.

Security is modelled in five layers — Organization-Wide Defaults, a role hierarchy, four profiles, a permission set and two sharing rules — so that a vendor manager, for example, can see event schedules without seeing client budgets. Three reports and a five-component dashboard provide operational visibility.

The system is implemented in a Developer Edition org for academic purposes, but the configuration follows production Salesforce practice throughout, and the Apex is bulk-safe and unit-tested.

---

1. Project Overview

EventForce automates and streamlines event management by connecting clients, vendors, venues and event coordinators within one integrated system. It covers the full lifecycle: client onboarding, event planning, vendor coordination, venue booking, cancellation approval and post-event feedback collection.

Salesforce was chosen because the majority of the requirement is configurable rather than codeable. Objects, relationships, page layouts, validation rules, approval processes, flows, security and reporting are all declarative, which keeps the implementation maintainable and lets the two genuinely programmatic needs — venue integrity and the nightly status sweep — be isolated in a small amount of Apex.

User roles

Role	Responsibility
Client	Requests event services, provides feedback, views event updates
Event Coordinator	Manages scheduling, client communication and approval workflows
Vendor	Handles assigned services — catering, decor, logistics
Venue Manager	Manages venue availability and reservations
Event Administrator	Oversees system operations, security and data monitoring

---

2. Objectives

1. Model clients, events, vendors, venues and feedback as a normalised Salesforce data model, including a many-to-many relationship between events and vendors.
2. Guarantee venue integrity in code — a venue must be reserved while booked and released when freed, and two live events must never occupy the same venue on the same date.
3. Automate client communication with a reminder three days before an event and a feedback request once it is completed.
4. Formalise event cancellation as an auditable approval process with status transitions and email notification to approver and client.
5. Enforce data quality at the point of entry through validation rules, and derive budget figures through formula and roll-up fields so they cannot be mistyped.
6. Restrict access by role so that commercially sensitive fields — client budget and vendor cost — are invisible to users who do not need them.
7. Provide operational insight through reports on upcoming events and vendor performance, consolidated into a single dashboard.
8. Keep the whole system maintainable by a non-programmer: five of the seven requirement areas are declarative.

3. Phase 1 — Requirement Analysis and Data Model

3.1 Custom objects

Object	API Name	Record Name	Purpose
Event	`Event__c`	Event Name (Text)	Central record; one row per engagement
Client	`Client__c`	Client Name (Text)	The party requesting services
Vendor	`Vendor__c`	Vendor Name (Text)	Service supplier
Venue	`Venue__c`	Venue Name (Text)	Physical location with capacity and availability
Feedback	`Feedback__c`	Feedback ID (Auto Number `FB-{0000}`)	Post-event rating and comments
Event Vendor	`EventVendor__c`	Event Vendor (Auto Number `EV-{0000}`)	Junction resolving Event ↔ Vendor

Full field listings, data types, picklist values and API names are in `docs/01-data-model.md`.

3.2 Relationships

From	To	Type	Rationale
`Event__c`	`Client__c`	Lookup	A client must survive the deletion of their events; events are optional for a client
`Event__c`	`Venue__c`	Lookup	An event may be held without a booked venue
`Feedback__c`	`Event__c`	Lookup	Feedback must not cascade-delete with the event it describes
`Feedback__c`	`Client__c`	Lookup	As above
`EventVendor__c`	`Event__c`	**Master-Detail**	Required for the roll-up summary on Event; junction rows

			are meaningless without a parent event
`EventVendor__c`	`Vendor__c`	**Master-Detail**	The second master-detail is what makes the object a junction

3.3 Entity-relationship summary

```
Client__c 1 ──────< N Event__c N >────── 1 Venue__c
    │                     │
    │                     │
    N                     1
    │                     │
Feedback__c N >────── 1 ──┘


Event__c 1 ──────< N EventVendor__c N >────── 1 Vendor__c
```

Event is the hub: it references one Client and one Venue by lookup, is referenced by many Feedback rows, and owns many EventVendor junction rows. Each junction row owns exactly one Vendor. The Event↔Vendor many-to-many is therefore resolved through `EventVendor__c`, which also carries `Estimated_Cost__c` — the per-assignment cost that the roll-up on Event aggregates.

> ***Screenshot:*** *Object Manager listing all six custom objects.* `images/01-object-manager.png`
>
> ***Screenshot:*** *Event object relationship diagram (Setup → Schema Builder, filtered to the six objects).* `images/02-schema-builder.png`

---

4. Phase 2 — Backend Development

4.1 Validation rules

Object	Rule	Formula	Effect
`Client__c`	`Email_Valid_Address`	`NOT(REGEX(Email__c, "^[a-zA-Z0-9._]+@[a-zA-Z0-9.]+\\.[a-zA-Z]{2,}$"))`	Blocks malformed email addresses with a field-level error
`Event__c`	`Event_Date_Not_In_Past`	`AND(ISNEW(), Event_Date__c < TODAY())`	New events cannot be backdated; existing records stay editable

Feedback__c	Rating_Within_Scale	OR(Rating__c < 1, Rating__c > 5)	Rating constrained to 1–5
Event__c	Budget_Must_Be_Positive	AND(ISNEW(), Event_Budget__c <= 0)	Budget must exceed zero

The double backslash in the email pattern is required because the formula parser consumes one backslash before the regular-expression engine sees the pattern.

4.2 Formula and roll-up fields

Object	Field	API Name	Type	Definition
Event__c	Vendor Cost	Vendor_Cost__c	Roll-Up Summary	SUM of EventVendor__c.Estimated_Cost__c
Event__c	Budget Balance	Budget_Balance__c	Formula (Currency)	Event_Budget__c - Vendor_Cost__c
Event__c	Budget Utilization	Budget_Utilization__c	Formula (Percent)	IF(Event_Budget__c > 0, Vendor_Cost__c / Event_Budget__c, 0)
Event__c	Days Until Event	Days_Until_Event__c	Formula (Number)	Event_Date__c - TODAY()
Feedback__c	Rating Label	Rating_Label__c	Formula (Text)	Nested IF mapping 1–5 to Very Poor … Excellent

Event_Budget__c remains an editable Currency field representing the agreed budget. Every other monetary figure is derived, so it cannot be entered inconsistently. The IF guard prevents division by zero when the budget is blank. Days_Until_Event__c supplies the entire filter condition for the upcoming-events report.

The roll-up summary is only possible because EventVendor__c relates to Event__c through master-detail. This is the concrete reason the junction object uses master-detail rather than lookups.

Screenshot: *Validation rule edit page showing the REGEX formula.* images/03-validation-rule.png

Screenshot: *Event record with Vendor Cost, Budget Balance and Budget Utilization populated from junction rows.* images/04-formula-fields.png

4.3 Apex

Six Apex files are in `apex/`. Two are application logic, three are tests and utilities, and one is the trigger itself.

`EventTrigger` on `Event__c` — before insert, before update, after insert, after update. The body contains no logic; it dispatches to `VenueAvailabilityHandler` based on `Trigger.isBefore`. This keeps the trigger thin, which is the standard one-trigger-per-object pattern and allows the behaviour to be disabled without editing trigger logic.

`VenueAvailabilityHandler` exposes two methods:

- `preventDoubleBooking(List<Event__c>, Map<Id, Event__c>)` — runs before the save. Queries all live events on the venues in scope, excluding the records currently being saved, and builds a set of `venueId|eventDate` keys. Any incoming record whose key already exists is rejected with `addError`. A second set catches two records within the same transaction claiming the same venue and date, which the query cannot see because neither row is committed yet.
- `updateVenueAvailability(List<Event__c>, Map<Id, Event__c>)` — runs after the save. Determines which venues should become `Reserved` and which should become `Available`, then confirms before releasing that no other live event still holds the venue. Only venues whose status actually differs are written, and the whole method issues one SOQL and one DML statement regardless of batch size.

Statuses that free a venue are `Canceled` and `Completed`. `Pending Cancellation` and `Rejected` deliberately do not free it: a cancellation under review, or one that has been refused, means the event is still going ahead.

`BatchCompleteEvents` implements `Database.Batchable<sObject>`. Its `start` method returns a `QueryLocator` over events whose date has passed and whose status is not `Completed`, `Canceled` or `Pending Cancellation`; `execute` sets each to `Completed` in chunks of 200; `finish` writes a debug message.

`ScheduleCompleteEvents` implements `Schedulable` and calls `Database.executeBatch(new BatchCompleteEvents(), 200)`. It exists so that Setup → Schedule Apex has a class to run on a timer. The job is configured as `Daily Event Completion`, weekly across all days, starting 20:00.

`VenueAvailabilityHandlerTest` contains twelve test methods covering reservation on insert, release on cancellation, release on completion, venue change, unrelated-field edits, double-booking rejection, same-transaction duplicate rejection, different-date acceptance, rebooking after cancellation, events with no venue, batch selectivity and scheduler execution.

Design points worth recording:

- **Bulk safety.** Both handler methods accumulate identifiers in sets and perform one query and one update per trigger invocation. No SOQL or DML appears inside a loop.

- **Null handling.** `null NOT IN ('Canceled','Completed')` evaluates to null rather than true, which would silently exclude events with a blank status. Both queries therefore use `(Event_Status__c = null OR Event_Status__c NOT IN :statuses)`.
- **Trigger context.** `addError` is only honoured in a before trigger, so validation runs before the save; the venue write affects a different object and must run after the save, once the event record certainly exists.
- **Interaction with the batch.** When the batch sets an event to `Completed`, that update fires the trigger, which releases the venue. The two components chain without additional configuration.

Screenshot: *Developer Console showing `VenueAvailabilityHandlerTest` with 12 passing tests and 100% coverage.* `images/05-apex-tests.png`

4.4 Automation — reminder flow

`Event 3-Day Reminder` is a **record-triggered flow with a scheduled path** on `Event__c`, configured to run after save and optimised for Actions and Related Records.

Entry condition: `Event Date` is not null. The scheduled path `3 Days Before Event` fires three days before `Event_Date__c` and contains four elements: a Get Records that re-reads the event (because the path executes days after it was scheduled and the status may have changed), a Decision that suppresses the email for `Canceled`, `Completed` or `Rejected` events and for events with no client, a second Get Records that retrieves the client, and a Send Email action addressed to `{!Get_Client.Email__c}`.

A record-triggered flow alone cannot satisfy this requirement, because it only executes when a record is saved and nothing saves an event three days before it occurs. The scheduled path is the mechanism that lets the platform compute a wake-up time from a date field. It runs once per record, and if the event date is edited the platform cancels the pending path and reschedules it, so reminders cannot duplicate.

Screenshot: *Flow Builder canvas showing the scheduled path and its four elements.* `images/06-flow-canvas.png`

Screenshot: *Paused and Waiting Interviews listing the queued `3 Days Before Event` path.* `images/07-flow-interview.png`

4.5 Automation — feedback notification flow

The project description requires automation for reminders, **feedback notifications** and approval updates. `Client Feedback Request` is a second record-triggered flow on `Event__c` that supplies the feedback notification.

Entry condition: `Event Status` equals `Completed`, with the record-triggered option set to run **only when a change is made in the record**. That setting is what makes the flow correct — without it, every subsequent save of a completed event would send the client a further email. With it, the flow fires exactly once, on the transition into `Completed`. The flow runs after save, because a Send Email action should not be part of the record's own transaction.

The flow contains three elements: a Get Records that retrieves the client (`$Record` exposes only fields of the triggering object, so the client's email must be fetched separately), a Decision that suppresses sending when the event has no client, and a Send Email action addressed to `{!Get_Client.Email__c}` requesting feedback.

This flow is significant because it completes an automated chain rather than operating in isolation. `BatchCompleteEvents` marks past events `Completed` each night; that update fires both `EventTrigger`, which releases the venues, and this flow, which asks the clients for feedback. A single scheduled job therefore produces venue maintenance and client communication with no manual step anywhere in the path.

> ***Screenshot:*** *Flow Builder canvas for `Client Feedback Request` showing the Get Records, Decision and Send Email elements.* `images/08-feedback-flow.png`

4.6 Automation — cancellation approval process

`Event Cancellation Approval` on `Event__c`. Entry criterion: `Event Status` equals `Planned`.

Stage	Actions
Initial submission	Field update `Set Status Pending Cancellation`; email alert to the approver using template `Cancellation_Request_Approver`, which includes `{!ApprovalRequest.Process_Url}` so the approver can act from the email
Approval step	`Event Coordinator Review`, single level, assigned to the Event Coordinator user
Final approval	Field update `Set Status Canceled`; email alert to the client's email address using template `Cancellation_Approved_Client`
Final rejection	Field update `Set Status Rejected`; email alert to the client using `Cancellation_Rejected_Client`; record unlocked for the owner
Locking	Record locked for editing while pending; unlocked on recall; submitters may recall

Three Classic email templates were created for this process. Because the final approval action sets the status to `Canceled`, `EventTrigger` fires and releases the venue automatically — the approval process and the Apex agree without any direct integration. A rejected request leaves the status at `Rejected`, which is not a venue-freeing status, so the venue correctly remains reserved.

Two limitations are acknowledged. Recall unlocks the record but does not revert the status, because approval processes expose no recall action; the status must be changed back manually. And a rejected event cannot submit a second request, because entry criteria require `Planned`.

Screenshot: *Approval process detail page showing all four action groups.* `images/09-approval-process.png`

Screenshot: *The received approver notification email.* `images/10-approval-email.png`

---

5. Phase 3 — UI Customisation

5.1 Tabs and application

Five custom tabs were created — Event, Client, Vendor, Venue and Feedback. `EventVendor__c` deliberately has no tab: junction records are managed from the *Event Vendors* related list on the Event record, and a tab would allow orphan rows to be created directly.

The Lightning application **Event Planner Management System** (API name `Event_Planner_Management_System`) groups those tabs with Home in the navigation bar, and adds Notes and Recent Items to the utility bar.

5.2 Page layouts

The Event page layout carries the derived budget fields, the related lists for Event Vendors, Feedbacks and Approval History, and the **Submit for Approval** action. Without that action on the layout the approval process cannot be started from Lightning Experience at all.

5.3 User experience outcomes

- Creating an event and selecting a venue immediately reserves that venue; the coordinator sees the effect without any manual step.
- Budget Balance and Budget Utilization update as vendor assignments are added, so overspend is visible during planning rather than after invoicing. Winter Music Festival in the sample data demonstrates an over-budget event at 107% utilisation.
- Feedback ratings render as words through `Rating_Label__c` rather than bare numbers.

Screenshot: *The application open in the App Launcher with the navigation bar visible.* `images/11-app-launcher.png`

Screenshot: *A complete Event record page showing related lists and derived fields.* `images/12-event-record.png`

---

6. Phase 4 — Data Migration and Sample Data

There was no legacy system to migrate from, so this phase consists of constructing a representative data set. It was implemented as `apex/SeedEventForceData.cls`, an Apex utility that creates 47 records in a single transaction: 4 clients, 4 venues, 6 vendors, 10 events, 22 vendor assignments and 5 feedback records.

The seeder was written in Apex rather than loaded through the Data Import Wizard for three reasons. It resolves the parent record identifiers programmatically, so relationships are guaranteed correct. All dates are computed relative to `Date.today()`, so the data set never ages out — one event is always exactly three days away for the reminder flow and three are always in the past for the batch. And it is repeatable: `wipe()` deletes every EventForce record in dependency order and `seed()` refuses to run while Event records exist, so the demo environment can be reset exactly.

The data set is constructed to exercise every feature:

Requirement	Data provided
Batch has work to do	Three past events left at status `Planned`
Batch status exclusion is provable	One past event at status `Canceled`
Reminder flow fires	One event dated today + 3 days
Approval process demo	Two `Planned` events that can be submitted, approved and rejected
Double-booking rule	Three live events sharing Grand Convention Hall on different dates, plus one cancelled event on the same venue
Over-budget reporting	Winter Music Festival at 1,600,000 committed against a 1,500,000 budget
Vendor performance spread	Six vendors across 22 assignments, totals from 365,000 to 1,525,000
Rating distribution	Feedback scores of 5, 4, 2, 5 and 3

> ***Screenshot:*** *The Venue list showing all four venues at* `Reserved` *after seeding.*
> *`images/13-seeded-venues.png`*

---

7. Phase 4 — Security

Five layers, each able to restrict but never to exceed the layer above it. Full configuration tables are in `docs/05-security.md`.

7.1 Organization-Wide Defaults

Object	OWD	Rationale
Client	Private	Personal contact data
Event	Private	Carries budgets and client relationships
Feedback	Private	Contains client criticism not intended for vendor-facing staff
Vendor	Public Read Only	Shared reference data; only vendor managers maintain it
Venue	Public Read Only	Availability must be visible to everyone planning an event
Event Vendor	Controlled by Parent	Automatic; a consequence of master-detail

7.2 Role hierarchy

```
Event Admin
├── Event Coordinator
├── Vendor Manager
└── Venue Manager
```

The hierarchy grants the Event Admin visibility of every record owned by the three subordinate roles. It grants nothing sideways — a Coordinator cannot see a Vendor Manager's records — which is the gap sharing rules fill. There is deliberately no Client role: clients are not internal staff, so their visibility derives from record ownership rather than a reporting line.

7.3 Profiles

Four profiles, each cloned rather than built from scratch: `Event Admin` from System Administrator, and `Event Coordinator`, `Vendor Manager` and `Client` from Standard User.

Object permissions, tab visibility, assigned applications and system permissions differ per profile. Two exclusions are deliberate: Vendor Manager has no access to the Client object at all, and Client has no access to the Vendor object.

7.4 Field-Level Security

Field	Event Admin	Event Coordinator	Vendor Manager	Client

Event_Budget__c	Read/Write	Read/Write	Hidden	Hidden
Vendor_Cost__c	Read	Read	Read	Hidden
Budget_Balance__c	Read	Read	Hidden	Hidden
Budget_Utilization__c	Read	Read	Hidden	Hidden
EventVendor__c.Estimated_Cost__c	Read/Write	Read/Write	Read/Write	Hidden

Budget_Balance__c and Budget_Utilization__c are hidden from Vendor Manager even though Vendor_Cost__c is visible, because both derive from Event_Budget__c. Field-Level Security is not propagated through formula evaluation, so a user who can see the balance and the vendor cost can recover the hidden budget by subtraction. Hiding the source field alone would therefore not protect it.

7.5 Permission set

Budget_Analysis grants read access to the four budget fields on Event. It is assigned to a single named Vendor Manager who occasionally reviews utilisation across events. Editing the profile would grant it to every vendor manager permanently; the permission set targets one user and can be revoked without touching the profile. This is the intended division of labour between the two mechanisms.

7.6 Sharing rules

Rule	Type	Criteria / Scope	Shared With	Access
Share Pending Cancellations with Admins	Criteria	Event Status = Pending Cancellation	Role: Event Admin	Read/Write
Coordinator Events to Vendor Managers	Role-based	Owner in role Event Coordinator	Role: Vendor Manager	Read Only

The first opens exactly the records an administrator must act on during a cancellation, and withdraws access automatically once the status changes. The second cannot be expressed through the role hierarchy, because Event Coordinator and Vendor Manager are siblings — hierarchy flows upward only.

7.7 Verification

All verification was performed through **Login As**. It cannot be performed as the administrator: System Administrator holds View All Data, which overrides Organization-Wide Defaults, sharing rules and record ownership simultaneously, so every restriction appears inert.

Screenshot: *Sharing Settings page showing the OWD table.* `images/14-owd.png`

Screenshot: *The role hierarchy tree.* `images/15-role-hierarchy.png`

Screenshot: *The same Event record as Event Admin (budget fields populated) and as Vendor Manager (budget fields blank).* `images/16-fls-comparison.png`

---

8. Phase 5 — Reporting and Analytics

8.1 Reports

A public report folder `EventForce Reports` holds three reports.

Report	Report Type	Grouping	Chart
Upcoming Events — Summary by Month	Events	Event Date, grouped by Calendar Month; filtered on `Days Until Event >= 0`	Vertical bar, count per month
Vendor Performance	Vendors with Event Assignments (custom)	Vendor Name; filtered on `Vendor Status = Active`; summed Estimated Cost	Horizontal bar by spend
Event Summary	Events	Event Status, nested Event Type; summed budget, average utilisation	Donut by status

`Vendors with Event Assignments` is a **custom report type** with Vendor as the primary object and Event Vendors added as a related child. It was necessary because `EventVendor__c` has no tab and therefore no automatically generated report type, and the standard Events report type cannot reach across the junction to vendor data.

8.2 Dashboard

`EventForce Dashboard` in the `EventForce Dashboards` folder, configured to run as the dashboard viewer, with five components:

Component	Type	Source
Upcoming Events by Month	Vertical bar	Upcoming Events — Summary by Month
Events by Status	Donut	Event Summary
Spend by Vendor	Horizontal bar	Vendor Performance

Events Over Budget	Table, filtered `Budget Balance < 0`	Event Summary
Total Upcoming Events	Metric	Upcoming Events — Summary by Month

Because the dashboard runs as the viewer, field-level security applies to it — a Vendor Manager opening the dashboard sees spend but not client budget.

> ***Screenshot:*** *Each of the three reports with its chart.* `images/17-report-upcoming.png`, `images/18-report-vendor.png`, `images/19-report-summary.png`
>
> ***Screenshot:*** *The complete dashboard with all five components populated.* `images/20-dashboard.png`

---

9. Phase 5 — Deployment, Maintenance and Documentation

9.1 Deployment strategy

The implementation lives in a Developer Edition org, so deployment here means reproducing the configuration in a target org rather than moving it between environments. Two routes are available.

The documented route is `docs/`: every org-side component is recorded with its exact settings, in build order, so the system can be reconstructed by hand in any org. This is the route used for assessment, because it doubles as evidence that each component is understood.

The tooling route is the Salesforce CLI. `sf project generate` creates a DX project, `sf project retrieve start` pulls the object, field, flow, validation rule, approval process, profile, permission set, sharing criteria and report metadata into `force-app/`, and `sf project deploy start` pushes it to a target org. The Apex in this repository is already in a paste-ready form and would sit unchanged under `force-app/main/default/classes` and `.../triggers`.

Sequencing matters in either route: objects before fields, fields before relationships, relationships before validation rules and roll-ups, and all of the above before flows, approval processes and profiles that reference them.

9.2 Maintenance

Activity	Frequency	Method
Apex job monitoring	Weekly	Setup → Apex Jobs; confirm `Daily Event Completion` completed without error
Flow interview monitoring	Weekly	Setup → Paused and Waiting Interviews; look for failed path

		interviews
Approval process review	Monthly	Setup → Approval Processes → view pending and completed requests
Test regression	After any Apex change	Run `VenueAvailabilityHandlerTest`; maintain 100% coverage on the handler
Data quality	Monthly	Run the three reports and investigate nulls in Client, Venue and Event Status
Security review	Each term	Login As each persona and confirm the expected fields and records are hidden

9.3 Troubleshooting record

The defects actually encountered during development, and their resolutions, are listed in `docs/06-demo-and-viva.md`. The three most significant:

1. The original double-booking rule rejected any second live event on a venue, which made the data model unusable — four venues could not carry ten events. Diagnosis came from reading the `addError` message against the seeded records. The rule was made date-aware.
2. The specified batch query, `Event_Status__c != 'Completed'`, also matched `Canceled` and `Pending Cancellation`, so the nightly job would have overwritten approval outcomes. Detected by diffing statuses before and after a batch run against seeded data. The query now excludes all three.
3. `Client__c.Email__c` was initially a Text field so that the custom validation rule would display. Email Alerts can only address Email-type fields, so the approval process could not notify the client. The field was converted to Email type and the validation rule's demonstration value changed to `john@college`, which passes Salesforce's native check but fails the custom regular expression.

9.4 Documentation summary

This report is maintained as Markdown in `PROJECT_REPORT.md` and rendered into the required submission format — Word and PDF, Times New Roman 12 pt, with a cover page, page header and `Page N of M` footer — by `tools/md_to_docx.py`. Generating the submission copy rather than typing it separately means the document can never drift away from the implementation notes it summarises.

Document	Contents

`README.md`	Repository orientation, Apex deployment order, recorded deviations from the specification
`docs/01-data-model.md`	Objects, fields, relationships, tabs, application
`docs/02-validation-and-formulas.md`	Validation rules, formula and roll-up fields
`docs/03-automation.md`	Apex behaviour, reminder flow, feedback notification flow, approval process
`docs/04-analytics-and-sample-data.md`	Seeder, report type, three reports, dashboard
`docs/05-security.md`	OWD, roles, profiles, field-level security, permission set, sharing rules, users
`docs/06-demo-and-viva.md`	Demo script, debugging log, viva question bank
`apex/`	Six Apex source files
`tools/md_to_docx.py`	Renders this report into the Word submission copy

---

10. Conclusion

EventForce demonstrates that the majority of a real CRM requirement is satisfied by configuration rather than code. Five of the seven functional areas — the data model, tabs and application, validation and derived fields, the reminder flow, the feedback notification flow and the approval process, and the entire security model — were built declaratively. Apex was needed in exactly two places, and both were cases where the platform offers no declarative equivalent: enforcing a cross-record uniqueness constraint across a venue and a date, and sweeping historical records on a schedule.

The most instructive part of the build was not any single component but the interaction between three of them. The batch job, the trigger and the approval process all write to `Event_Status__c`, and the system only behaves correctly once all three agree on which statuses free a venue and which do not. `Canceled` and `Completed` release the venue; `Pending Cancellation` and `Rejected` must not, because in both cases the event is still going ahead. Getting that set wrong produced the two most significant defects found during development — the batch overwriting approval outcomes, and venues released while a cancellation was merely under review.

Two findings are worth carrying forward. Field-Level Security is not propagated through formula evaluation, so protecting a field means also protecting every formula derived from it. And a restricted security model cannot be tested by an administrator, because `View All Data` overrides every layer beneath it — verification requires impersonating each persona.

The system is complete against the specification, unit-tested at 12 passing test methods, and reproducible from `docs/` alone. Extending it to production would need multi-day events, so the booking check tests date overlap rather than equality, and the status constants moved into custom metadata so they can be changed without a deployment.

Appendix A — Screenshots required

Captured during the demo and inserted at the placeholders above.

File	Content
`01-object-manager.png`	Object Manager listing all six custom objects
`02-schema-builder.png`	Schema Builder showing the relationship map
`03-validation-rule.png`	`Email_Valid_Address` edit page with the REGEX formula
`04-formula-fields.png`	Event record with the three derived budget fields populated
`05-apex-tests.png`	Developer Console test run, 12 passing
`06-flow-canvas.png`	Flow Builder canvas with the scheduled path
`07-flow-interview.png`	Paused and Waiting Interviews showing the queued path
`08-feedback-flow.png`	`Client Feedback Request` Flow Builder canvas
`09-approval-process.png`	Approval process detail page with all four action groups
`10-approval-email.png`	The received approver notification email
`11-app-launcher.png`	Application tile in the App Launcher
`12-event-record.png`	Complete Event record page
`13-seeded-venues.png`	Venue list after seeding
`14-owd.png`	Sharing Settings OWD table
`15-role-hierarchy.png`	Role hierarchy tree
`16-fls-comparison.png`	Same event as Admin and as Vendor Manager
`17-report-upcoming.png`	Upcoming Events — Summary by Month
`18-report-vendor.png`	Vendor Performance

`19-report-summary.png`	Event Summary
`20-dashboard.png`	EventForce Dashboard

Appendix B — Component inventory

Component	Count	Names
Custom objects	6	`Event__c`, `Client__c`, `Vendor__c`, `Venue__c`, `Feedback__c`, `EventVendor__c`
Custom fields	24	Including 1 roll-up and 5 formulas
Relationships	6	4 lookup, 2 master-detail
Custom tabs	5	Event, Client, Vendor, Venue, Feedback
Lightning apps	1	Event Planner Management System
Validation rules	4	1 required, 3 additional
Apex classes	5	Handler, batch, schedulable, test, seeder
Apex triggers	1	`EventTrigger` on `Event__c`
Test methods	12	All passing
Flows	2	Event 3-Day Reminder (1 scheduled path), Client Feedback Request
Email templates	3	Approver request, client approved, client rejected
Email alerts	3	One per template
Field updates	3	Pending Cancellation, Canceled, Rejected
Approval processes	1	Event Cancellation Approval, 1 step
Scheduled Apex jobs	1	Daily Event Completion, 20:00
Profiles	4	Event Admin, Event Coordinator, Vendor Manager, Client

Roles	4	Event Admin plus 3 subordinates
Permission sets	1	Budget_Analysis
Sharing rules	2	1 criteria-based, 1 role-based
Report types	1 custom	Vendors with Event Assignments
Reports	3	Upcoming Events, Vendor Performance, Event Summary
Dashboards	1	EventForce Dashboard, 5 components